Embedded Real Time Systems

Toolchain Migration Journal

Course Vivado 2017.2 / SDK to native Vivado and Vitis 2026.1

Firas Harbo Saleh Student ID: 202109174

Review snapshot: `assignment_2/Firas`, commit `c38a216`.

Purpose. Record the differences that affect this assignment, the post-review changes, why they were made, and remaining acceptance boundaries. This is an observed project migration, not a benchmark of all Vivado releases or a substitute for course approval.

1. Starting point and acceptance boundary

The course assignment explicitly requires Vivado 2017.2 (p. 2), and its installation guide includes Software Development Kit (SDK). The supplied Ex7 project also records 2017.2 and the original ZYBO part `xc7z010clg400-1`. Our implementation uses installed native Vivado/Vitis 2026.1 on Ubuntu 26.04 LTS, targeting that same part.

The installation guide was tested on Windows 10 but offers a Windows or Linux installer. Linux is therefore not itself a violation of the written guide; using 2026.1 instead of 2017.2 is the explicit version deviation. A successful board run does not establish instructor acceptance of that deviation.

Layer	Course / original project	Current implementation
Hardware design	Vivado 2017.2 project and board preset	Vivado 2026.1 project/IP upgrade, explicit constraints
HLS	Vivado HLS, guided GUI steps	Vitis HLS, Tcl-controlled Vivado IP flow
Hardware handoff	SDK export using HDF	Fixed XSA including bitstream
Embedded application	SDK workspace and legacy BSP	Vitis Unified platform, standalone domain, generated SDT BSP
Automation	SDK-era project/debug workflow	Vitis Python build plus XSDB Tcl board launch
Peripheral lookup	Legacy device-ID configuration	Base-address/compatible-name configuration where adapted
ANN register interface	One original AXI-Lite slave	Two generated slaves: control and array storage
Physical target	Original ZYBO XC7Z010	Same ZYBO; no Z7 board substitution

Evidence conventions

Observed changes have local source/artifact evidence. **Toolchain changes** enable the new workflow; **independent corrections** address algorithms or robustness. The reviewed commit is the starting point; this revision includes subsequent local source fixes.

No 2017.2 toolchain was rerun. Historical comparisons use the course material and supplied artifacts, not a new performance comparison between releases.

2. HLS and hardware-project changes

Scripted HLS with a controlled baseline

`run_hls.tcl` creates baseline and pipelined solutions for the same XC7Z010 part and 10 ns target. Both explicitly use `-flow_target vivado` so the output is reusable IP, rather than a Vitis acceleration kernel flow. Both set `config_compile -pipeline_loops 0`; only the pipelined solution adds II=1 directives to the two named inner dot-product loops.

AMD's migration documentation records changes in automatic loop-pipelining and clock-uncertainty defaults between historical Vivado HLS and Vitis HLS [W3]. Explicit settings make our two-solution comparison interpretable; a tool upgrade alone must not be mistaken for the effect of a deliberate pipeline. The current reports show 2.70 ns clock uncertainty. This is an observed 2026.1 report value, not a reconstructed 2017.2 setting.

The flow performs C simulation and synthesis for both solutions, VHDL co-simulation for the lower-latency solution, and IP-catalog export with `-rtl vhdl -version 1.0.1`. The export log and settings retain the requested version; IP XML represents it as version 1.0 plus core revision 1. Reading the XML's version field alone would incorrectly suggest the request was ignored.

Project, IP and directory migration

The original Ex7 XPR names the board preset `digilentinc.com:zybo:part0:1.0`. The migrated XPR keeps the silicon part but no longer depends on that unavailable local preset. The block design changes from the original XML representation to a 2026.1 JSON representation, and the processing-system, GPIO, interconnect and reset IP revisions change.

`build_ex7_2026.tcl` selects the newly exported HLS-IP repository, sets the button GPIO to a custom four-bit input interface, attempts the IP upgrades, connects the switch/LED ports and new array bus, assigns addresses, validates the BD, generates targets and runs implementation through bitstream generation. It writes timing/utilization reports and a bitstream-containing XSA.

The script uses the constraints already stored in the project; it does not generate those pin assignments itself. The change from board automation to explicit ports/pins is a practical repair for this machine's missing old board preset, not a rule that every 2026.1 project must avoid board files.

Migration is not merely opening an old project

Accepting an upgrade dialog updates project metadata, but does not prove IP compatibility, correct pins, usable BSP drivers or application correctness. For this project the integration repairs, new BSP, driver configuration and physical UART check were separate necessary steps. Directory layout changes and renamed generated files also mean that old paths in SDK-era instructions cannot simply be copied unchanged.

For Exercises 3–5, a direct upgrade encountered locked legacy board-preset outputs. The new `build_ex345_2026.tcl` reconstructs the supplied Ex5 block design from its original Tcl using current catalog revisions, unchanged custom LED/matrix RTL, and explicit ZYBO pins. This Ex3-style processor/IO platform with added matrix IP hosts three separate applications.

3. Software platform, drivers and board connections

HDF / SDK to XSA / Vitis Unified

The original retained Vivado log copies a system definition to HDF and launches SDK. The new hardware script uses `write_hw_platform -fixed -include_bit` to produce `Ex7_2026_1.xsa`. The native Python builder creates a standalone platform/domain for `ps7_cortexa9_0`, imports the application, shared timer header and AMD platform support, and builds the ELF.

The important distinction is between project automation and debugging. AMD states that XSCT is no longer available starting with 2026.1 and recommends the Python APIs for project management; XSDB remains available for debug [W1]. Accordingly our project build uses Vitis Python, while our working board launch uses Tcl XSDB. These are complementary tools, not contradictory workflows.

SDT lookup semantics

The newer system-device-tree (SDT) configuration no longer identifies devices with the old `DeviceId` field; compatible names and base addresses are used [W2]. The shared timer selects the base-address lookup under `SDT` and retains the old device-ID path otherwise. Ex3 now applies this distinction to the switch GPIO. Separately, custom LED-IP access uses register 0, not an AXI GPIO driver. Semantic generated macros select peripherals without relying on arbitrary numbered GPIO instances. Ex345 LEDs are at `0x43C10000`, unlike the original Ex3-only `0x43C00000` mapping.

Two ANN AXI-Lite buses and a project-specific workaround

The supplied old IP has one AXI-Lite interface. Our current generated IP splits control and array storage, despite the shared source bundle name. The saved BD and generated driver establish the following mapping:

Interface	Function	Base address
`s_axi_BUS`	Start/done/control	`0x43C00000`
`s_axi_BUS_r`	Input, weights and output storage	`0x40000000`
`buttons/S_AXI`	AXI GPIO	`0x41200000`

The generated SDT config table lists the ANN addresses in ascending order, but the driver fields expect control first and arrays second. The application's `initializeAccelerator` constructs the named fields explicitly: `Bus_BaseAddress = BASEADDR_1` and `Bus_r_BaseAddress = BASEADDR`. This is verified for this generated artifact. New compile-time/runtime guards check these project-specific addresses and verify that driver memory widths/depths match the shared dimension header.

Physical pins stay tied to the actual board

LED0–3 remain M14/M15/G14/D18; SW0–3 remain G15/P15/W13/T16. The XDC also activates existing button pins under the generated port name. The generated VHDL continuously connects switches to LEDs. Correct constraints and numerical inference output do not replace the required physical LED test.

4. Changes that are not caused by the tool version

These changes should be credited as implementation work, not presented as unavoidable Vivado migration requirements.

Change	Reason and consequence
Weight row strides	Uses `INPUT_LEN*i` for input weights and `NEURONS_LEN*i` for output weights. These are the contiguous fully connected layouts, unlike the overlapping rows in the starter reference.
48-bit accumulators	Protect fixed-point dot-product accumulation before shifts, particularly the second layer. This is arithmetic design, not an IDE compatibility patch.
Extended testbench	Preserves the original; adds an explicitly adapted course vector and four signed/zero/overflow cases with an independent integer reference.
Aligned transfer buffers	Packs byte/short arrays into word-aligned buffers before driver calls. Avoids relying on casts from differently aligned arrays.
Transfer and polling checks	Checks word counts and bounds ready/done polling. Makes failures visible rather than permitting an indefinite wait.
Separated measurements	Retains CPU/load/run timers; adds full packing-to-readback latency and ten-trial min/mean/max at application `-O2`.

The reference discrepancy must be visible

A direct host build of the untouched supplied testbench against the current core exits 1. Output 0 is 331 versus 41.003418 in that reference; all ten outputs disagree. The current custom test passes, and a temporary 8/3/4-dimension host variant also passes. These facts can coexist because the tests describe different weight layouts and arithmetic references.

The implemented remedy is a documented course-derived adapted reference, preserving the original and identifying changed strides and fixed-point expectations. Default and 8/3/4 host suites pass; the untouched reference still fails. Acceptance of the adaptation remains a course decision. Neither silently replacing the reference nor changing the core to overlapping weight rows merely to satisfy that reference is a defensible migration story.

Historical pre-fix performance, with the correct interpretation

Baseline/pipelined HLS latency is 33,056/6,612 cycles; both are 2026.1 builds. VHDL co-simulation measures 6,610 cycles. Pipelining increases DSP from 2 to 3, FF from 638 to 1,112 and LUT from 716 to 1,126; BRAM18K stays at 10.

The fresh board run reports CPU 186,747 ticks, loading 233,849 ticks and start-to-done 21,700 ticks. At 325 MHz these are 574.6, 719.5 and 66.8 μs. The 8.61× compute-only comparison is against an `-O0 -g3` ARM application, not optimized CPU performance. The load-plus-run subtotal is 255,549 ticks; packing, setup and output readback are outside it. Loading alone already exceeds software compute, demonstrating the one-shot transfer bottleneck.

5. Reproducing the native workflow safely

The commands below document the revised native flow. Run each stage sequentially, with only one hardware/build session using the project at a time.

Before rebuilding: preserve canonical sources outside generated folders. HLS anchors its working directory and renames the old project as a backup; Vivado resets synthesis. Ex7 Vitis reuses its workspace with source backups, or renames it on explicit `-rebuild`. Ex345 also preserves old workspaces.

Stage A: regenerate HLS IP

From the repository root `ERTS_assigments`:

```
source /home/firas/AMD/2026.1/Vitis/settings64.sh
vitis-run --mode hls --tcl \
  Assignment_2/exercise7_neural_net/hls/run_hls.tcl
```

Check both C simulations, synthesis tables and VHDL co-simulation. Confirm the generated IP exists before updating the hardware project.

Stage B: implement and export hardware

From the same repository root, using a separate shell for Vivado settings:

```
source /home/firas/AMD/2026.1/Vivado/settings64.sh
vivado -mode batch -source \
  Assignment_2/hardware/build_ex7_2026.tcl
```

Check BD validation, implementation completion, timing and bitstream/XSA creation. This is a batch process, not a reason to open another GUI. The current scripts cap implementation jobs and tool threads at two.

Stage C: build the standalone application

From the repository root in the Vitis-configured shell:

```
vitis -s Assignment_2/exercise7_neural_net/sdk/build_vitis_2026.py
```

Check the generated address map, ANN config fields and compiler flags, then confirm `Assignment_2/hardware/Ex7_vitis/neuralNetTest/build/neuralNetTest.elf`.

Stage D: run on the identified ZYBO

Reuse an existing `hw_server` on port 3121. Identify the expected cable and XC7Z010 first, then capture UART at 115200 baud and run:

```
python3 Assignment_2/hardware/capture_board.py 7 \
  --output ../outputs/fix_evidence/board_ex7_new_run
```

This resets and programs volatile board state, initializes the PS, downloads the ELF and starts the CPU; it does not write flash. The launcher selects the known cable and rejects absent/ambiguous targets. Real UART bytes, timestamped events, XSDB output and artifact hashes are retained, not reconstructed images.

6. Provenance, remaining work and sources

Commit	Observed role
`47e3b21`	Initial exercise implementation, fixed-point core, tests and software logic.
`29f6373`	Main 2026.1 Ex7 hardware-project migration and build flow.
`cbec41c`	Native Vitis build, SDT compatibility, dual-bus address handling, transfer hardening and board launch.
`c38a216`	Recorded successful physical benchmark; reviewed repository snapshot.

Changes implemented after review

1. Correct custom LED-IP access and SDT GPIO lookup; explicitly select an OCM linker script and verify the built ARM ELF heap/stack locations.
2. Restore prescribed matrix types/signatures and literal transpose inputs, retaining the previous fixture as a regression case.
3. Preserve the original neural reference and add a documented adapted course vector plus signed, zero-state and wide-accumulation tests.
4. Build applications at `-O2`; report ten ANN trials and full transaction timing alongside the separate loading and compute measurements.
5. Share dimensions across C/C++, round transfer buffers up to whole words, zero padding, and reject stale generated driver dimensions at compile time.
6. Preserve workspaces, guard addresses, select the cable serial and retain artifact hashes with raw UART evidence; update stale build instructions.

These are local post-review fixes. The user confirmed the full physical LED counter during the 11 October rerun. Separate switch mapping observations and course acceptance of 2026.1 and the adapted reference remain outstanding. The hand-in and review closure addendum record the latest completed verification runs.

Local evidence

Course PDFs: `work/assignment2/Assignment_2/ERTS_Assignment2_ANN_ZYBO.pdf` and `/home/firas/Downloads/ERTS_Vivado_Installation_2017_2.pdf`. Original project: `/home/firas/Downloads/Ex7.xpr.zip`; SHA-256: `6928f168497e39fdb69fe8ebe2b08604eae389b49132aec17fc6949fabaf4cac`.

Under `ERTS_assigments/Assignment_2/`, primary implementation evidence is `hardware/build_ex7_2026.tcl`, `hardware/Ex7/Ex7.xpr`, its saved `system.bd`/XDC, `exercise7_neural_net/hls/run_hls.tcl`, `exercise7_neural_net/sdk/build_vitis_2026.py`, `exercise7_neural_net/sdk/neuralNetTest.c`, and `common/zynq_timer.h`. Generated `xneuralnet_g.c`, header and `xparameters.h` substantiate the bus-field workaround. Historical physical evidence is under `outputs/native_screenshots/zybo_retry_20261011T085514Z/`; its blank screenshot attempt is excluded. Post-review build and UART evidence is under `outputs/fix_evidence/`, including the recorded user counter observation.

AMD primary references (accessed 11 October 2026)

[W1] UG1400, 2026.1: XSCT to Python API Migration. Supports project-automation changes and continued XSDB debugging.

[W2] UG1647, 2024.2: What's New in the SDT-based flow. Supports device-ID versus compatible-name/base-address semantics.

[W3] UG1399, 2021.1: Default User Control Settings. Historical migration context for HLS defaults; current numerical settings are taken from this project's 2026.1 reports, not inferred from this older page.